Universidade Federal de Ouro Preto - UFOP

Instituto de Ciências Exatas e Biológicas - ICEB

Departamento de Computação - DECOM

Ciência da Computação

Trabalho Prático 1 - Pesquisa externa

BCC203 - Estrutura de Dados 2

Arlon Gabriel G. Machado, Ives Henrique S. de Oliveira, Leonardo Junio I. da S. Martins, Tiago Carneiro de Castro.

Professor: Guilherme Tavares de Assis

Ouro Preto
15 de outubro de 2026

Sumário

1 Introdução **1**
1.1 Considerações Iniciais 1
1.2 Especificiações da máquina 1

2 Desenvolvimento **1**
2.1 Acesso sequencial indexado 1
2.2 Árvore binária de pesquisa 1
2.3 Árvore B 2
2.4 Árvore B* 2

3 Análise dos testes **3**
3.1 Critérios de medição e seleção das chaves 3
3.2 Comparação do custo de construção 4
3.3 Desempenho das pesquisas em arquivos ordenados 4
3.4 Influência da ordem de entrada 5
3.5 Diferenças entre as árvores B e B* 5
3.6 Interpretação dos tempos e do uso de memória 6
3.7 Construção e conjunto de consultas 6
3.8 Abrangência e limitações dos resultados 6

4 Conclusão **7**

5 Ferramentas de apoio ao desenvolvimento **7**

Lista de Tabelas

1 Resultados do acesso sequencial indexado 1
2 Resultados da árvore binária de pesquisa 2
3 Resultados da árvore B 2
4 Resultados da árvore B* 3

1 Introdução

Este trabalho prático, tem como objetivo implementar e testar diferentes métodos de pesquisa em arquivos como Acesso Sequencial Indexado, árvore binária de pesquisa, árvore B e árvore B*. A proposta é entender como cada método funciona e observar suas diferenças na hora de buscar informações.

Para isso, serão realizados testes com diferentes quantidades de dados, analisando o tempo gasto nas buscas, o número de comparações e a quantidade de vezes que os arquivos precisam ser acessados. Com esses resultados, será possível comparar os métodos e entender quais apresentam melhor desempenho em cada situação.

1.1 Considerações Iniciais

Ambiente de desenvolvimento do código: Visual Studio Code
Sistema operacional: Windows 11
Liguagem: C
Ambiente de desenvolvimento da documentação: Latex Overleaf.

1.2 Especificiações da máquina

Intel Core I9 13700H com 16GB de RAM e SSD nvme.

2 Desenvolvimento

Este trabalho compara quatro métodos de pesquisa externa implementados em C. Os testes registrados utilizam arquivos de 100, 200, 2.000 e 20.000 registros, nas ordens crescente, decrescente e aleatória, quando aplicáveis. Cada configuração pesquisa 20 chaves distintas e existentes. A construção do índice ocorre uma vez; as médias correspondem às pesquisas. A geração dos dados, a seleção das chaves e a impressão ficam fora das medidas.

2.1 Acesso sequencial indexado

A função criarIndicePaginas percorre o arquivo, verifica sua ordenação e guarda em RAM a primeira chave de cada página de quatro registros. Em pesquisarPagina, a procura é sequencial tanto no índice quanto na página escolhida. O método aceita arquivos crescentes e decrescentes. Arquivos aleatórios não se aplicam.

Em todos os tamanhos testados, a construção fez n leituras e n 1 comparações. Cada pesquisa leu quatro registros, mas as comparações médias passaram de 16,65 para 2.501,10 com o crescimento do arquivo. Esse aumento decorre da varredura linear do índice. O cálculo da última página exige cuidado para permitir a leitura mesmo quando ela está completa.

Tabela 1: Resultados do acesso sequencial indexado

Registros	Ordem	LC	CC	LP	CP
100	Crescente	100	99	4,00	16,65
100	Decrescente	100	99	4,00	16,65
200	Crescente	200	199	4,00	28,90
200	Decrescente	200	199	4,00	28,90
2.000	Crescente	2.000	1.999	4,00	253,60
2.000	Decrescente	2.000	1.999	4,00	253,60
20.000	Crescente	20.000	19.999	4,00	2.501,10
20.000	Decrescente	20.000	19.999	4,00	2.501,10

LC e CC: leituras e comparações na construção. LP e CP: médias de leituras e comparações por pesquisa, considerando 20 chaves distintas e existentes.

2.2 Árvore binária de pesquisa

Os nós ficam em um arquivo separado e armazenam a chave, a posição do registro e as posições dos filhos esquerdo e direito. A inserção preserva a ordem de entrada. Na busca, o programa lê os nós

do caminho e recupera o registro completo no arquivo de dados ao encontrar a chave. As posições numéricas substituem ponteiros de RAM no índice externo.

As entradas ordenadas formam uma cadeia, com construção quadrática. Para 2.000 registros, foram 2.001.000 leituras de construção, contra 25.539 na ordem aleatória. As leituras médias de pesquisa caíram de 1.000,05 para 13,45. Em 20.000 registros ordenados, a construção chegou a 200.010.000 leituras. O percurso iterativo evita que essa cadeia consuma a pilha de chamadas.

Tabela 2: Resultados da árvore binária de pesquisa

Registros	Ordem	LC	CC	LP	CP
100	Crescente	5.050	4.950	51,55	100,10
100	Decrescente	5.050	9.900	51,55	100,10
100	Aleatória	913	1.278	10,20	17,40
200	Crescente	20.100	19.900	101,55	200,10
200	Decrescente	20.100	39.800	101,55	200,10
200	Aleatória	1.796	2.315	10,40	17,80
2.000	Crescente	2.001.000	1.999.000	1.000,05	1.997,10
2.000	Decrescente	2.001.000	3.998.000	1.000,05	1.997,10
2.000	Aleatória	25.539	34.802	13,45	23,90
20.000*	Crescente	200.010.000	199.990.000	9.990,05	19.977,10
20.000*	Decrescente	200.010.000	399.980.000	9.990,05	19.977,10

LC e CC: leituras e comparações na construção. LP e CP: médias de leituras e comparações por pesquisa, considerando 20 chaves distintas e existentes.

* Resultados das execuções manuais registradas nas imagens.

2.3 Árvore B

A árvore B mantém em RAM apenas referências com chave e posição do registro. A ordem definida no código é 50, com até 100 referências por página. A inserção divide páginas cheias e promove uma referência ao pai; a pesquisa percorre as chaves da página sequencialmente. Depois de localizar a referência, o programa lê o registro completo no arquivo.

A construção realizou n leituras em todas as configurações, e cada pesquisa bem-sucedida fez uma leitura. Com 20.000 registros, as comparações médias de pesquisa ficaram entre 53,95 e 86,10. As divisões alteram a distribuição das chaves, por isso essas médias não crescem de forma contínua. Na implementação, a propagação das divisões e a criação de uma nova raiz exigem cuidado.

Tabela 3: Resultados da árvore B

Registros	Ordem	LC	CC	LP	CP
100	Crescente	100	5.148	1,00	51,50
100	Decrescente	100	5.246	1,00	51,40
100	Aleatória	100	5.334	1,00	45,65
200	Crescente	200	13.240	1,00	40,80
200	Decrescente	200	13.438	1,00	42,15
200	Aleatória	200	13.218	1,00	38,25
2.000	Crescente	2.000	193.132	1,00	45,45
2.000	Decrescente	2.000	163.234	1,00	52,00
2.000	Aleatória	2.000	169.921	1,00	54,60
20.000	Crescente	20.000	3.013.553	1,00	72,95
20.000	Decrescente	20.000	1.723.371	1,00	53,95
20.000	Aleatória	20.000	2.239.281	1,00	86,10

LC e CC: leituras e comparações na construção. LP e CP: médias de leituras e comparações por pesquisa, considerando 20 chaves distintas e existentes.

2.4 Árvore B*

A implementação separa páginas internas e folhas. As internas armazenam chaves separadoras e filhos; as folhas guardam referências aos registros no arquivo. A pesquisa sempre chega a uma folha, mesmo quando encontra a chave em uma página interna. Na igualdade com um separador, segue o filho da direita, conforme o modelo do professor.

Assim como na árvore B, a construção leu n registros e cada pesquisa bem-sucedida leu um registro completo. Para 20.000 registros, as comparações médias de pesquisa variaram de 52,95 a 76,55, abaixo da árvore B nas três ordens. A divisão da folha deve conservar as referências e copiar a primeira chave da folha direita para o índice superior.

Tabela 4: Resultados da árvore B*

Registros	Ordem	LC	CC	LP	CP
100	Crescente	100	5.148	1,00	51,50
100	Decrescente	100	5.246	1,00	51,40
100	Aleatória	100	5.334	1,00	45,65
200	Crescente	200	13.243	1,00	41,05
200	Decrescente	200	13.343	1,00	41,65
200	Aleatória	200	13.167	1,00	37,95
2.000	Crescente	2.000	193.171	1,00	48,85
2.000	Decrescente	2.000	161.411	1,00	51,95
2.000	Aleatória	2.000	168.721	1,00	57,30
20.000	Crescente	20.000	2.999.832	1,00	65,20
20.000	Decrescente	20.000	1.689.455	1,00	52,95
20.000	Aleatória	20.000	2.219.746	1,00	76,55

LC e CC: leituras e comparações na construção. LP e CP: médias de leituras e comparações por pesquisa, considerando 20 chaves distintas e existentes.

3 Análise dos testes

A análise considera separadamente a construção dos índices e a pesquisa, pois essas etapas apresentam custos diferentes. A construção organiza as referências necessárias ao método, enquanto a pesquisa utiliza a estrutura já montada para recuperar um registro. Essa separação permite identificar quando o custo está na preparação dos dados e quando aparece novamente em cada consulta.

Os resultados abrangem arquivos com 100, 200, 2.000 e 20.000 registros. Para os três primeiros tamanhos, foram avaliados os quatro métodos nas situações aplicáveis. Com 20.000 registros, há resultados do acesso sequencial indexado nas duas ordens, das árvores B e B* nas três ordens e da árvore binária nas ordens crescente e decrescente. Isso corresponde a 43 configurações e 860 pesquisas de chaves existentes.

3.1 Critérios de medição e seleção das chaves

Em cada configuração, o programa constrói o índice uma vez e realiza 20 pesquisas. Portanto, os valores de construção representam uma execução dessa etapa; os valores de pesquisa são médias de 20 consultas distintas. Dividir o tempo de construção por 20 produziria um custo amortizado por consulta, que é uma medida diferente do tempo necessário para montar a estrutura.

A seleção das chaves utiliza uma posição em cada um de 20 intervalos disjuntos do arquivo. Esse procedimento distribui a amostra ao longo das posições dos registros e evita concentrar todas as pesquisas em uma região pequena. Nos arquivos aleatórios, essa distribuição corresponde às posições no arquivo, sem garantir espaçamento uniforme entre os valores numéricos das chaves. A semente fixa permite repetir a seleção no mesmo ambiente, e as saídas dos lotes utilizam as mesmas chaves entre os métodos para cada combinação de quantidade e situação.

A geração do arquivo, a seleção das chaves e a impressão dos resultados ficam fora dos tempos dos métodos. Durante as consultas do experimento, os arquivos necessários já estão abertos. A pesquisa contabiliza o percurso do índice e a leitura do registro encontrado; a abertura inicial dos arquivos de pesquisa e a liberação final das estruturas ficam fora desse intervalo.

As leituras são contadas por item transferido pelo código. Uma chamada de `fread` que retorna quatro registros acrescenta quatro leituras. Na árvore binária, a leitura de um nó e a leitura de um registro completo também incrementam esse contador. Como esses itens têm tamanhos diferentes, o número de leituras informa a quantidade de transferências lógicas, mas não mede diretamente os bytes transferidos nem os acessos físicos ao dispositivo de armazenamento.

3.2 Comparação do custo de construção

O acesso sequencial indexado, a árvore B e a árvore B* apresentaram a mesma quantidade de leituras de construção: uma por registro do arquivo. Com 20.000 registros, cada um realizou 20.000 leituras. A diferença está nas operações feitas depois de receber cada registro. O indexado verifica a ordem e guarda a primeira chave de cada página; B e B* precisam localizar a posição de inserção, deslocar referências e dividir páginas quando necessário.

Por isso, a igualdade de leituras não corresponde à igualdade de comparações. Com 20.000 registros crescentes, o indexado realizou 19.999 comparações, a árvore B realizou 3.013.553 e a B* realizou 2.999.832. Apesar dessa diferença, os tempos de construção foram, respectivamente, 0,120197, 0,104247 e 0,106652 segundo. Esse resultado mostra que contar comparações isoladamente não permite prever o tempo total: as comparações dos índices B e B* acontecem em RAM, e o tempo também inclui leitura, alocação, chamadas de funções e outras operações não representadas por esse contador.

Na árvore binária, as entradas ordenadas produzem um custo muito maior. Como cada novo registro percorre a cadeia formada pelos anteriores, a quantidade de leituras de construção pode ser expressa, para as chaves distintas desses arquivos, por:

$$L_{\text{construcao}} = n + \frac{n(n-1)}{2} = \frac{n(n+1)}{2}.$$

O termo n corresponde às leituras dos registros de dados, e o restante corresponde às leituras dos nós visitados nas inserções. A fórmula resulta em 5.050 leituras para 100 registros, 20.100 para 200, 2.001.000 para 2.000 e 200.010.000 para 20.000, exatamente como registrado nos testes. Ao aumentar de 2.000 para 20.000 registros, a entrada cresce dez vezes e as leituras crescem aproximadamente cem vezes.

As duas ordens geram cadeias de igual comprimento, mas a contagem de comparações na construção é diferente. Na ordem crescente, a primeira condição da inserção identifica que a nova chave é maior e escolhe o filho direito. Na decrescente, essa condição é falsa e o programa também verifica se a chave é menor. Assim, foram 199.990.000 comparações na crescente e 399.980.000 na decrescente para 20.000 registros. Essa diferença decorre das condições executadas no código, enquanto a quantidade de nós percorridos e de leituras permanece igual.

A construção da árvore binária também grava os novos nós e atualiza os filhos dos pais. Essas escritas entram no tempo medido, embora o contador solicitado no trabalho registre apenas leituras. Nos testes manuais de 20.000 registros, a construção levou 308,256127 segundos na ordem crescente e 317,032146 segundos na decrescente, enquanto os outros métodos ordenados ficaram próximos de um décimo de segundo. O custo quadrático da cadeia é, portanto, a diferença mais acentuada dessa etapa.

3.3 Desempenho das pesquisas em arquivos ordenados

No acesso sequencial indexado, todas as pesquisas registradas leram quatro registros, independentemente do tamanho do arquivo. Os tamanhos testados são múltiplos da capacidade da página, de modo que inclusive a última página contém quatro registros. Esse comportamento mantém pequena a quantidade de dados recuperada por consulta, enquanto a procura sequencial na tabela de índices cresce com o número de páginas.

As comparações médias do indexado passaram de 16,65, com 100 registros, para 28,90, com 200, 253,60, com 2.000, e 2.501,10, com 20.000. O arquivo de 20.000 registros produz uma tabela com 5.000 entradas. Como a amostra cobre diferentes regiões do arquivo, a pesquisa percorre uma parte considerável dessa tabela antes de selecionar a página. Assim, quatro leituras por consulta não significam um custo total constante: a parcela de comparações do índice continua crescendo.

As ordens crescente e decrescente apresentaram as mesmas médias de leituras e comparações no indexado. O código inverte a relação usada para percorrer as chaves do índice, e as amostras ocupam posições equivalentes nos arquivos gerados. A diferença de tempo entre as ordens não está acompanhada de uma diferença nessas operações contabilizadas.

A árvore binária ordenada apresenta crescimento tanto das leituras quanto das comparações de pesquisa. As leituras médias foram 51,55, 101,55, 1.000,05 e 9.990,05 nos quatro tamanhos. Como cada nó da cadeia fica no arquivo, a consulta precisa recuperar sucessivamente os nós do caminho antes de ler o registro completo. Para uma busca bem-sucedida que visita v nós, o código contabiliza $v + 1$

leituras e $2v - 1$ comparações: verifica a igualdade em cada nó e a direção em cada nó anterior ao encontrado.

Com 20.000 registros crescentes, a binária realizou 9.990,05 leituras e 19.977,10 comparações por pesquisa, contra quatro leituras e 2.501,10 comparações do indexado. B e B* fizeram uma leitura e, respectivamente, 72,95 e 65,20 comparações. Nessa configuração, o indexado fez cerca de 34 vezes as comparações da B, enquanto a binária acumulou o percurso da cadeia em memória externa. As diferenças decorrem da organização das estruturas e do local em que cada comparação é realizada.

3.4 Influência da ordem de entrada

A ordem aleatória modificou fortemente os resultados da árvore binária. Com 2.000 registros, as leituras de construção caíram de 2.001.000, nas entradas ordenadas, para 25.539. Isso representa aproximadamente 78 vezes menos leituras. As comparações de construção foram 34.802 na aleatória, contra 1.999.000 na crescente e 3.998.000 na decrescente. A distribuição das inserções entre os dois filhos reduziu os caminhos percorridos naquele arquivo.

A diferença também apareceu nas consultas. Para 2.000 registros, a binária passou de 1.000,05 leituras e 1.997,10 comparações médias nas ordens crescente e decrescente para 13,45 leituras e 23,90 comparações na aleatória. O tempo médio caiu de aproximadamente 1.319 e 1.306 μs para 32,455 μs. A entrada aleatória apresentou, portanto, um resultado muito melhor que as cadeias ordenadas. Esse comportamento depende da disposição das chaves: a implementação da árvore binária não executa operações que garantam seu balanceamento.

Mesmo nessa situação favorável, a binária continuou mais lenta que B e B*. Com 2.000 registros aleatórios, ela fez 23,90 comparações por pesquisa, enquanto B fez 54,60 e B* fez 57,30. Entretanto, a binária precisou de 13,45 leituras, e as outras duas fizeram uma. Seus tempos médios foram 32,455 μs, 7,790 μs e 8,135 μs, respectivamente. A quantidade menor de comparações da binária coexiste com um custo maior de recuperação dos nós no arquivo.

B e B* aceitaram as três ordens e preservaram o balanceamento da estrutura por divisão de páginas. A ordem ainda alterou a distribuição das chaves, os deslocamentos nas inserções e os caminhos pesquisados. Com 20.000 registros, a B realizou 3.013.553 comparações de construção na crescente, 1.723.371 na decrescente e 2.239.281 na aleatória. Contudo, seus tempos foram 0,104247, 0,105617 e 0,113199 segundo. A decrescente teve menos comparações que a crescente e um tempo ligeiramente maior, reforçando que o contador não representa sozinho todo o trabalho executado.

O acesso sequencial indexado não foi aplicado aos arquivos aleatórios. A primeira chave de cada página permite selecionar o intervalo correto quando existe ordenação. Esse cenário é não aplicável ao método, diferentemente dos cenários aplicáveis que ainda não possuem resultados.

3.5 Diferenças entre as árvores B e B*

Na versão testada, a ordem definida para as duas árvores é 50, com capacidade máxima de 100 referências por página da B e 100 referências por folha da B*. Portanto, os resultados não decorrem de uma capacidade maior das folhas da B*. A distinção está na organização: B pode encontrar uma referência em uma página interna, enquanto B* utiliza separadores internos e continua até uma folha.

Com 100 registros, ambas cabem em uma única página e apresentaram contagens idênticas em cada ordem. Na crescente, por exemplo, foram 5.148 comparações de construção e 51,50 comparações médias de pesquisa em cada árvore. As diferenças de tempo desse caso não correspondem a diferenças nos contadores analisados. Com 200 registros, as divisões já produzem diferenças, mas nenhum método apresenta vantagem uniforme nas comparações de todas as ordens.

Com 20.000 registros, a B* realizou menos comparações de construção nas três situações, mas as reduções foram inferiores a 2%. Na pesquisa, suas médias foram 65,20, 52,95 e 76,55 comparações nas ordens crescente, decrescente e aleatória, contra 72,95, 53,95 e 86,10 da B. Essa vantagem nos contadores não se repetiu no tempo das consultas: a B apresentou 6,595, 6,350 e 5,875 μs, enquanto a B* apresentou 8,155, 6,595 e 6,095 μs.

Na construção de 20.000 registros, a B foi ligeiramente mais rápida na crescente. A B* foi mais rápida na decrescente e na aleatória. Nesta última, foram 0,091341 segundo para B* e 0,113199 segundo para B, uma redução observada de aproximadamente 19,3% no tempo de construção. A classificação depende, assim, da etapa e da situação: o menor número de comparações, o menor tempo de construção e o menor tempo de pesquisa não pertencem necessariamente ao mesmo método.

3.6 Interpretação dos tempos e do uso de memória

Os tempos não aumentaram de forma contínua em todos os casos. Na ordem crescente, o indexado registrou 0,072569 segundo de construção para 100 registros e 0,021236 para 200. Na pesquisa, a B registrou 9,430 μs com 100 registros e 6,595 μs com 20.000, embora o número médio de comparações tenha passado de 51,50 para 72,95. Essas inversões mostram que as medidas registradas incluem variação de execução além do crescimento das operações do algoritmo.

Buffers de leitura, cache do sistema operacional, escalonamento e atividades simultâneas podem influenciar esses tempos. O experimento não isolou a contribuição de cada fator, de modo que eles são explicações possíveis para a variação, sem permitir atribuir uma diferença específica a uma causa medida. A construção e a seleção das chaves também acessam o arquivo antes das consultas e podem deixar dados em cache, mesmo quando suas operações ficam fora do cronômetro de pesquisa.

As médias correspondem a 20 chaves diferentes, com caminhos potencialmente distintos. Elas descrevem a amostra selecionada e não representam 20 repetições da mesma consulta nem uma média de todas as chaves do arquivo. Sem repetições completas dos cenários e medidas de dispersão, diferenças pequenas entre B e B* devem ser interpretadas como vantagens observadas nessa execução. A impressão de nove casas decimais também não garante precisão experimental nessa escala.

A localização dos índices ajuda a interpretar as leituras. B e B* percorrem referências em RAM e só leem o registro completo quando encontram a chave. O indexado também consulta uma tabela em RAM, mas lê a página inteira escolhida. A binária mantém seu índice em arquivo e contabiliza a leitura dos nós do caminho. Uma leitura por pesquisa em B e B* expressa essa configuração do trabalho; uma implementação com as páginas dessas árvores armazenadas em disco teria outro custo de transferência.

Os registros completos permanecem no arquivo de dados, incluindo o campo de 5.000 caracteres. Os índices guardam chaves e posições, evitando manter esse campo em cada item da estrutura em RAM. O consumo efetivo de memória, porém, não foi medido. Também não foram avaliadas outras ordens para B e B* nem outras capacidades de página para o indexado. As comparações de desempenho correspondem aos parâmetros usados neste TP.

3.7 Construção e conjunto de consultas

Para avaliar o custo do experimento como uma tarefa, é possível somar o tempo de construção aos tempos das 20 pesquisas:

$$T_{\text{etapas}} = T_{\text{construcao}} + 20\,\overline{T}_{\text{pesquisa}}.$$

Essa soma utiliza somente as etapas cronometradas e mantém fora a geração, a seleção das chaves, a impressão e as demais operações não incluídas nesses intervalos.

Com 2.000 registros crescentes, a soma resulta em aproximadamente 0,041403 segundo para o indexado, 7,921020 segundos para a binária, 0,010649 segundo para B e 0,012731 segundo para B*. Nesse conjunto de 20 consultas, a construção pesa muito mais no resultado do que as diferenças de microssegundos entre as pesquisas de B e B*.

A possibilidade de reutilizar o índice altera essa relação. Com mais consultas sobre o mesmo arquivo, o custo de construção é distribuído por uma quantidade maior de pesquisas. Se a estrutura precisar ser reconstruída frequentemente, a preparação volta a ter maior peso. No experimento, essa reutilização ocorre dentro das 20 consultas; cada execução do programa monta novamente o índice utilizado.

3.8 Abrangência e limitações dos resultados

As 20 chaves existentes foram encontradas em cada configuração registrada. Esse resultado verifica as consultas da amostra, mas não demonstra o funcionamento para todas as chaves possíveis, arquivos com repetições, pesquisas sem sucesso ou arquivos incompletos. Como os testes analisados são de buscas bem-sucedidas, suas médias também não descrevem o custo de procurar uma chave inexistente.

Os ensaios de 200.000 e 2.000.000 registros exigidos pelo enunciado ainda não foram executados. Também falta a árvore binária com 20.000 registros aleatórios. A ausência desses resultados limita a avaliação do crescimento em arquivos maiores e da viabilidade dos índices em memória. Os dados da binária ordenada com 20.000 registros são de execuções manuais complementares ao lote automatizado.

Os dois lotes automatizados possuem identificadores distintos dos executáveis utilizados. Essa diferença não informa, por si só, se houve alteração no algoritmo, na compilação ou apenas nos dados incorporados ao executável. Os registros também não documentam suficientemente as características do equipamento e o estado do sistema para reproduzir com precisão os tempos. Uma comparação temporal mais controlada exige repetir os cenários com o mesmo executável, condições documentadas e medidas de dispersão.

4 Conclusão

Os testes mostraram que a ordem dos registros influencia fortemente a árvore binária externa: entradas ordenadas formaram cadeias que elevaram o custo de construção e pesquisa. O acesso sequencial indexado manteve quatro leituras por consulta, enquanto suas comparações aumentaram com o tamanho do índice.

As árvores B e B* apresentaram baixo custo de pesquisa, utilizando índices em RAM e uma leitura para recuperar cada registro encontrado. Com 20.000 registros, a B apresentou os menores tempos de consulta, enquanto a B* teve construção mais rápida nas ordens decrescente e aleatória. Assim, a escolha do método depende da organização dos dados e da importância de cada etapa.

Os principais cuidados de implementação envolvem as posições dos registros em arquivo, as divisões de páginas e a contabilização das operações. Para completar a avaliação exigida, ainda faltam os testes com 200.000 e 2.000.000 registros e a árvore binária com 20.000 registros aleatórios.

5 Ferramentas de apoio ao desenvolvimento

Durante o desenvolvimento deste trabalho, foram utilizadas ferramentas de Inteligência Artificial (IA) como apoio em algumas etapas, principalmente para esclarecer dúvidas e auxiliar na organização do projeto.

O uso dessas ferramentas ocorreu na criação do arquivo ‘README.md‘, no esclarecimento de dúvidas específicas sobre estruturas de dados mais complexas, na identificação de possíveis melhorias no código, como a atualização de variáveis e a adoção de práticas mais atuais de programação, além de algumas revisões pontuais do código.

A IA também foi utilizada como auxílio na elaboração do relatório, especialmente na organização das informações e na construção de tabelas para apresentação dos resultados dos testes.

Referências

[1] ASSIS, Guilherme Tavares de. [Ouro Preto]: Universidade Federal de Ouro Preto. Slides da disciplina Estrutura de Dados II (BCC203), em PDF. Material disponibilizado no Moodle da UFOP. Acesso em: 13 de Outubro 2026.